

P8X Instruction Set -- Quick Reference (rev D)

Opcodes, mnemonics and cycle counts generated live from genucode.py (the microcode source of truth) -- cannot drift from the hardware.

Operands: #imm immediate | addr 16-bit absolute | (Pn) ptr indirect | (Pn)+ post-increment | (no operand) implied. **By**=bytes, **Cy**=cycles (incl. fetch). **Flags (F)**: C carry (active-high: ADD carry-out / SUB,CMP no-borrow A>=B), Z zero, N negative (bit7), V overflow. '.' = none. Signed branches test N^V / Z.

Memory (rev E): \$0000-1FFF ROM | \$2000-FEFF RAM | \$FF00-FFFF I/O. P0=PC, P3=stack (empty-descending). JZ/JNZ/JC are aliases of BZ/BNZ/BCP.

System						Stack					
Op	Mnemonic	By	Cy	FI	Description	Op	Mnemonic	By	Cy	FI	Description
\$00	NOP	1	2	-	No operation.	\$70	PHA	1	2	-	Push A onto P3 stack.
\$01	HLT	1	2	-	Halt clock; resume only by reset.	\$71	PLA	1	3	ZN	Pop A from P3 stack.
\$72	CLC	1	2	C	C=0.	16-bit memory (rev D)					
\$73	SEC	1	2	C	C=1.	\$74	PHW addr	3	10	-	Push 16-bit word at addr (hi then lo: lies little-endian at P3+1).
Interrupts (rev C)						\$75	PLW addr	3	12	-	Pop 16-bit word into addr (lo then hi).
\$02	EI	1	2	-	Enable maskable interrupts (IE=1).	\$76	LPW1 addr	3	9	-	P1 = 16-bit word at addr.
\$03	DI	1	2	-	Disable maskable interrupts (IE=0).	\$77	LPW2 addr	3	9	-	P2 = 16-bit word at addr.
\$04	RTI	1	10	-	Return from interrupt: pop flags then PC; re-enable IE.	\$78	MOVW dst,src	5	13	-	16-bit mem->mem: word at src -> dst.
\$08	IRQ	1	10	-	SW interrupt: push PC+flags, vector to \$0808.	\$79	LPW3 addr	3	9	-	P3 = 16-bit word at addr (restore a saved SP).
Load / store						Tier A: C-compiler ISA (2026-09, pure microcode; A! = clobbers A)					
\$10	LDA #imm	2	2	ZN	A:=immediate.	\$38	LDP1 #imm16	3	5	-	P1:=imm16 (3 bytes; was the LPL1/LPH1 pair).
\$11	LDB #imm	2	2	ZN	B:=immediate.	\$39	LDP2 #imm16	3	5	-	P2:=imm16.
\$12	LDA addr	3	6	ZN	A:=byte at addr (absolute).	\$3A	LDP3 #imm16	3	5	-	P3:=imm16.
\$13	LDB addr	3	6	ZN	B:=byte at addr (absolute).	\$3C	ADDP3 #imm	2	6	CZN	P3:=P3+imm8 (free a frame). A!; flags from the low byte.
\$14	STA addr	3	6	-	byte at addr:=A (absolute).	\$3D	SUBP3 #imm	2	6	CZN	P3:=P3-imm8 (allocate a frame). A!; flags from the low byte.
\$15	LDA (P1)+	1	2	ZN	A:=[P1], P1++.	\$90	LDW addr, (P1+d)	14	CZN	word at addr:=word at P1+d. A!	
\$16	LDA (P2)+	1	2	ZN	A:=[P2], P2++.	\$91	LDW addr, (P2+d)	14	CZN	word at addr:=word at P2+d. A!	
\$17	LDA (P3)+	1	2	ZN	A:=[P3], P3++.	\$92	LDW addr, (P3+d)	14	CZN	word at addr:=word at P3+d (local -> memory word). A!	
\$1A	STA (P2)+	1	2	-	[P2]:=A, P2++.	\$94	STW (P1+d), addr	14	CZN	word at P1+d:=word at addr. A!	
\$1B	STA (P3)+	1	2	-	[P3]:=A, P3++.	\$95	STW (P2+d), addr	14	CZN	word at P2+d:=word at addr. A!	
\$1D	STA (P1)	1	2	-	[P1]:=A.	\$96	STW (P3+d), addr	14	CZN	word at P3+d:=word at addr (memory word -> local). A!	
\$1E	STA (P2)	1	2	-	[P2]:=A.	\$98	LDW addr, #imm8	8	-	word at addr:=imm8 zero-extended (4 bytes).	
\$1F	STA (P3)	1	2	-	[P3]:=A.	\$99	LDW addr, #imm16	9	-	word at addr:=imm16 (5 bytes).	
\$51	LDA (P1)	1	2	ZN	A=[P1] (P1 kept).	\$9A	ADDW dst,src	5	15	CZNV	word a=a+b, 16-bit; C=carry out. A!; Z from the high byte only.
\$52	LDA (P2)	1	2	ZN	A=[P2] (P2 kept).	\$9B	SUBW dst,src	5	15	CZNV	word a=a-b, 16-bit; C=1 no borrow (a>=b unsigned). A!; Z high byte only.
\$53	LDA (P3)	1	2	ZN	A=[P3] (P3 kept).	\$9C	CMPW dst,src	5	15	CZNV	flags from a-b (16-bit), memory unchanged: C=unsigned a>=b, BLT/BGE = signed. A!
\$88	LDA (P1+d)	2	7	CZN	A:=byte at P1+d (d unsigned 0..255). C from the address add, Z/N from A.	\$9E	INCW addr	3	9	CZN	word at addr += 1. A!; flags from the low byte.
\$89	LDA (P2+d)	2	7	CZN	A:=byte at P2+d.	\$9F	DECW addr	3	9	CZN	word at addr -= 1. A!; flags from the low byte.
\$8A	LDA (P3+d)	2	7	CZN	A:=byte at P3+d (a stack local).	\$A0	ADDW addr, #imm8	11	CZNV	word a=a+imm8 (zero-ext), 16-bit, C=carry out. A!	
\$8C	STA (P1+d)	2	9	CZN	byte at P1+d:=A. A kept; flags clobbered by the address add.	\$A1	SUBW addr, #imm8	11	CZNV	word a=a-imm8, C=1 no borrow. A!	
\$8D	STA (P2+d)	2	9	CZN	byte at P2+d:=A.	\$A2	CMPW addr, #imm8	11	CZNV	flags from a-imm8 (16-bit), memory unchanged. A!	
\$8E	STA (P3+d)	2	9	CZN	byte at P3+d:=A (a stack local).	\$A4	LEAW addr, (P1+d)	14	CZN	word at addr:=P1+d (the address of a frame local). A!	
ALU (A,B -> A)						\$A5	LEAW addr, (P2+d)	14	CZN	word at addr:=P2+d. A!	
\$20	ADD	1	2	CZN	A:=A+B.	\$A6	LEAW addr, (P3+d)	14	CZN	word at addr:=P3+d (address of a stack local). A!	
\$21	SUB	1	2	CZN	A:=A-B.	Control flow					
\$22	AND	1	2	CZN	A:=A AND B.	\$40	JMP addr	3	5	-	P0(PC):=addr.
\$23	OR	1	2	CZN	A:=A OR B.	\$41	JSR (P1)	1	9	-	Push return addr, P0:=P1.
\$24	XOR	1	2	CZN	A:=A XOR B.	\$42	RTS	1	7	-	Pop return addr from P3 into P0.
\$25	CMP	1	2	CZN	Flags from A-B; A unchanged.	\$43	JSR addr	3	13	-	Push return addr, P0:=addr.
\$26	INC	1	2	CZN	A:=A+1 (B unused).	\$48	BZ addr	3	5	-	Branch if Z=1. (JZ alias.)
\$27	DEC	1	2	CZN	A:=A-1 (B unused).	\$49	BNZ addr	3	5	-	Branch if Z=0. (JNZ alias.)
\$28	SHL	1	2	CZN	A:=A<<1, 0->bit0, out->C.	\$4A	BCP addr	3	5	-	Branch if C=1 / A>=B unsigned. (JC alias.)
\$29	SHR	1	2	CZN	A:=A>>1, 0->bit7, out->C.	\$4C	JNC addr	3	5	-	Branch if C=0 / A<B unsigned.
\$2A	ROL	1	2	CZN	Rotate A left through carry.	Signed branches (rev C; after CMP)					
\$2B	ROR	1	2	CZN	Rotate A right through carry.	\$44	BLT addr	3	5	-	Branch if signed A<B (N^V=1). After CMP.
ALU with T (rev C; 2nd operand=T, B preserved)						\$45	BGE addr	3	5	-	Branch if signed A>=B (N^V=0). After CMP.
\$80	ADDT	1	2	CZN	A:=A+T (B preserved).	\$46	BLE addr	3	5	-	Branch if signed A<=B (N^V)/Z). After CMP.
\$81	SUBT	1	2	CZN	A:=A-T (B preserved).	\$47	BGT addr	3	5	-	Branch if signed A>B. After CMP.
\$82	ANDT	1	2	CZN	A:=A AND T (B preserved).	Pointer registers					
\$83	ORT	1	2	CZN	A:=A OR T (B preserved).	\$31	LPL1 #imm	2	3	-	P1 low byte:=imm.
\$84	XORT	1	2	CZN	A:=A XOR T (B preserved).	\$32	LPL2 #imm	2	3	-	P2 low byte:=imm.
\$85	CMPT	1	2	CZN	Flags from A-T; A,B unchanged.	\$33	LPL3 #imm	2	3	-	P3 low byte:=imm.
\$86	LDT #imm	2	2	-	T:=immediate.	\$35	LPH1 #imm	2	3	-	P1 high byte:=imm.
\$87	LDT addr	3	6	-	T:=byte at addr (absolute).	\$36	LPH2 #imm	2	3	-	P2 high byte:=imm.
						\$37	LPH3 #imm	2	3	-	P3 high byte:=imm.
						\$54	INP1	1	2	-	P1:=P1+1.
						\$55	INP2	1	2	-	P2:=P2+1.
						\$56	INP3	1	2	-	P3:=P3+1.
						\$58	DEP1	1	2	-	P1:=P1-1.
						\$59	DEP2	1	2	-	P2:=P2-1.
						\$5A	DEP3	1	2	-	P3:=P3-1.
						\$5E	TAP1L	1	2	-	P1 low:=A.
						\$5F	TAP1H	1	2	-	P1 high:=A.
						\$60	TAP2L	1	2	-	P2 low:=A.
						\$61	TAP2H	1	2	-	P2 high:=A.
						\$62	TAP3L	1	2	-	P3 low:=A.
						\$63	TAP3H	1	2	-	P3 high:=A.
						\$68	TPA1L	1	2	ZN	A:=P1 low.
						\$69	TPA1H	1	2	ZN	A:=P1 high.
						\$6A	TPA2L	1	2	ZN	A:=P2 low.
						\$6B	TPA2H	1	2	ZN	A:=P2 high.
						\$6C	TPA3L	1	2	ZN	A:=P3 low.
						\$6D	TPA3H	1	2	ZN	A:=P3 high.